

Skill 3 · Referência — Acesso a dados

Versão: v1.10 · Referência da Skill 3 · Progressive disclosure

Skill interna — carregar **somente** quando o núcleo (`skill-03-base-conversao-lsp-java.md`) indicar esta âncora. Aplique as regras globais do Router.

Acesso a dados (após API-first)

```
Tabela USU_* custom?  
SIM → IEntity + .sc + ICursor (EntitySession)  
NÃO (R* nativa / USU_* em tabela nativa) → ContextSession SELECT ou DBCenter DML  
Readonly em com.senior.rh.entities.readonly.*? → importar, não recriar
```

Fatal: `.sc` apontando para `Rxxxxx` → "Tabela R034FUN inválida. É permitido mapear somente tabelas de usuário".

ICursor — uma linha

```
MappedParamProvider params = new MappedParamProvider();  
params.setParam("vNumEmp", vNumEmp);  
ICursor<IMinhaTabela> cur =  
getContainer().getEntitySession().newCursor(IMinhaTabela.class);  
cur.addFilter("USU_NumEmp = :vNumEmp", params);  
cur.open();  
try {  
    if (cur.first()) { IMinhaTabela row = cur.read(); }  
} finally { cur.close(); }
```

ICursor — múltiplas linhas + ordem

```
String[] campos = new String[]{"USU_DatAlt"};  
OrderDirection[] ordem = new OrderDirection[]{OrderDirection.DESC};  
cur.setOrder(campos, ordem);  
cur.open();  
try {  
    while (cur.next()) {  
        IMinhaTabela row = cur.read();  
    }  
} finally { cur.close(); }
```

Três formas de EntitySession

```
// 1) preferencial no execute()  
getContainer().getEntitySession().newCursor(T.class);  
  
// 2) auxiliar fora do execute()
```

```

IEntitySession es = EntitySessionProvider.getSession();
es.newCursor(T.class);

// 3) chave exata (CursorUtil)
ICursor<T> cur = CursorUtil.getCursor(T.class);
T buffer = cur.newBuffer();
buffer.setNumEmp(col.getNumEmp());
cur.open();
try {
    if (cur.search(buffer, SearchMode.EXACT_MATCH)) { cur.read(buffer); }
} finally { cur.close(); }

```

ContextSession (SELECT) vs DBCenter (DML)

	ContextSession	DBCenter
Uso	SELECT	INSERT/UPDATE/DELETE
Close	Não fechar (container)	Fechar no finally
Placeholder	?	?
Índice ResultSet	base-0 (getInt(0) = 1º campo)	—
databaseId	—	ex. "vetorh" (confirmar projeto)

DBCenter INSERT (sanitizado)

```

IDBCenter database = DBCenter.getInstance("vetorh"); // confirmar databaseId do
projeto
ISession session = null;
try {
    session = database.newSession();
    Date dataConv =
Date.from(dataPro.atStartOfDay(ZoneId.systemDefault()).toInstant());
    String dataStr = new SimpleDateFormat("dd/MM/yyyy").format(dataConv);
    session.executeUpdate(
        "INSERT INTO USU_TExemplo (USU_NumEmp, USU_Dat, USU_CodSit) VALUES (?, ?,
?)",
        numEmp, dataStr, codSit);
} catch (Exception ex) {
    ctx.mensagemLog("Erro DML: " + ex.getMessage());
} finally {
    if (session != null) session.close();
}

```

Template IEntity (só USU_*)

```

package gestaopontoCustom; // package do projeto – sanitizar

import java.time.LocalDate;

```

```

import com.senior.dataset.IEntity;
import com.senior.dataset.annotation.Entity;
import com.senior.dataset.annotation.Field;

@Entity
public interface IMinhaTabela extends IEntity {

    @Field(description = "Empresa")
    int getUSU_CodEmp();
    void setUSU_CodEmp(int USU_CodEmp);
    boolean isUSU_CodEmpNull();
    void setUSU_CodEmpNull();

    @Field(description = "Data")
    LocalDate getUSU_DatEmi();
    void setUSU_DatEmi(LocalDate USU_DatEmi);
    boolean isUSU_DatEmiNull();
    void setUSU_DatEmiNull();

    @Field(description = "Usuário - long")
    long getUSU_CodUsu();
    void setUSU_CodUsu(long USU_CodUsu);
    boolean isUSU_CodUsuNull();
    void setUSU_CodUsuNull();
}

```

Null: `int v = entity.isUSU_CodEmpNull() ? 0 : entity.getUSU_CodEmp();`

IDs de usuário/consultor → **long**, não **int**.

Import correto: `com.senior.dataset.IEntity` — **nunca** `com.senior.g5...IEntity`.

Norma capitalização `USU_*`: getters/setters/null **espelham o nome do campo** exatamente como no banco/ `.sc` (ex.: campo `USU_CodEmp` → `getUSU_CodEmp` / `setUSU_CodEmp`). Não “javaizar” para `getUsuCodEmp`. Divergência só com evidência do schema do projeto.

Template `.sc` (JSON puro)

```

{
  "datasets": [
    {
      "id": "ScMinhaTabela",
      "entity": "gestaopontoCustom.IMinhaTabela",
      "storage": {
        "databaseId": "vetorh",
        "query": "from USU_MINHATAB USU_MINHATAB",
        "fieldMapping": [
          { "field": "USU_CodEmp", "column": "USU_MINHATAB.USU_CodEmp" },
          { "field": "USU_DatEmi", "column": "USU_MINHATAB.USU_DatEmi" },
          { "field": "USU_CodUsu", "column": "USU_MINHATAB.USU_CodUsu" }
        ],
        "type": "DB"
      }
    }
  ]
}

```

```
    }  
  ]  
}
```

Regras: `id` = nome do arquivo sem extensão; começa com `{` ; sem BOM/comentários; só `USU_*` no query .

Readonly do framework (importar)

`IR030EMP` , `IR060DSI` , `IR004HOR` , `IR010SIT` , `IR010TOB` , `IR014SIN` , `IR070ACC` , ...

Getters encadeados / `ISeparacaoHoras` (`padrao_compilacao`)

```
int codSin = ctx.getHistoricoSindicato().getCodSin();  
int tabOrg = ctx.getHistoricoLocal().getTabOrg();  
int numLoc = ctx.getHistoricoLocal().getNumLoc();  
HistoricoVinculo hv = ctx.getHistoricoVinculo();  
if (hv != null) { /* usar hv */ }  
  
ISeparacaoHoras sep = ctx.getHorasSeparadas(TipoIntervalo.REFEICA0);  
int diu = sep.getHorasDiurnas();  
int not = sep.getHorasNoturnas();  
int tot = sep.getTotalHoras();
```

Relacionados

Núcleo Skill 3 · Skill 1 · Skill 2 · Skill 5